데이터과학 파이썬

심송용(한림대학교 데이터과학스쿨)

http://jupiter.hallym.ac.kr

statistics 모듈

statistics 모듈

statistics 모듈은 파이썬의 내장 모듈로 일부 기초통계량을 계산해준다. 이 모듈에는

- mean(), fmean(): 둘다 산술평균을 계산한다. fmean 함수는 데이터를 float로 변환한 후 계산하여 좀더 효율적이 계산이 가능하다. fmean은 3.8버전 이후.
- geometric_mean(), harmonic_mean(): 각각 기하평균과 조화평균을 계산한다.
- median(), median_low(), median_high(): 중앙값을 계산한다. 자료가 짝수 개 있으면 median 함수는 오름차순으로 나열한 자료에서 중앙에 위치한 두 자료의 평균을, median_low 함수는 자료이 중앙의 두 값 중 왼쪽 값을, median_high는 오른쪽 값을 반환한다.
- median_grouped(): 중앙값은 50% 되는 점인데 이 값이 계산에는 여러 가지 보간(interpolation)이 사용된다. 이 함수는 이 보간법을 사용한 중앙값을 반환한다.
- mode(), multimode(): 각각 최빈값을 계산하되 multimode 함수는 둘 이상의 최빈값은 모두 반환한다. multimode는 파이썬 3.8 이후
- quantiles(): 분위수를 계산한다.

statistics 모듈

- pstdev(), pvariance(): 각각 모집단 버전의 표준편차와 분산을 반환한다. (분모 = n)
- stdev(), variance(): 각각 표본 버전의 표준편차와 분산을 반환한다.(분모 = $n-1$)

```
# statisticstest1.py
import statistics # 생략 가능

x = [i for i in range(1,11)]
print(statistics.mean(x))    ⇨ 5.5
print(statistics.fmean(x))    ⇨ 5.5
print(statistics.median(x))    ⇨ 5.5
print(statistics.median_low(x))  ⇨ 5
print(statistics.median_high(x))  ⇨ 6
print(statistics.median_grouped(x))  ⇨5.5
print(statistics.pvariance(x)) ⇨ 8.25
print(statistics.variance(x)) ⇨ 9.16667
```

statistics 모듈

```
print(statistics.pstdev(x)) ⇨ 2.87228
print(statistics.stdev(x))  ⇨ 2.02765
```

```
x = ['red', 'red', 'red', 'blue', 'blue', 'blue']
print(statistics.mode(x))  ⇨ red
print(statistics.multimode(x)) ⇨  ['red', 'blue']
```

numpy 패키지

numpy 패키지(넘파이 또는 넘피로 읽음)는 파이썬의 패키지 중의 하나로 공식 사이트는 https://numpy.org/
이며 수치계산을 위한(특히 행렬, 벡터 등)을 위한 필수적이 패키지로 다음의 로로를 사용한다.

numpy는 파이썬 베이스 모듈에는 없으므로 import를 반드시 해야 한다. numpy를 import 할 때는 관례적으로 np라는 이름으로 import를 많이 한다. 즉,
import numpy as np
로 많이 사용된다. 이하 np가 나오면 import 된 numpy를 의미한다.

벡터 및 행렬연산(ndarray: n-dimensional array)

ndarray: numpy에서 array 클래스를 사용하여 1차원 벡터, 2차원 행렬 및 그 이상의 고

numpy 패키지

차원 배열을 만들 수 있는데 이를 ndarray라고 한다. ndarray라는 이름의 클래스나 메소드가 존재하는 것이 아니고 numpy내의 배열을 ndarray라고 부른다(ndarray = d-dimensional array)

벡터의 생성

벡터(행렬등 배열 포함)는 array 메소드를 사용하여 원소를 나열하여 생성할 수 있다. ndarray의 연산은 원소대 원소로 이루어진다. 즉, x와 y가 길이가 같을 때 x+y와 x*y 등은 x[i]+y[i], x[i]*y[i]로 이루어진다. 이는 파이썬의 리스트 등에서 이루어지는 연산과 차이가 있으므로 주의하여야 하며 R-언어의 연산 방식을 따른다. 이 연산방식은 행렬을 포함한 2차원 이상에서도 같은 방식으로 사용한다. 이러한 연산을 위해서는 두 ndarray의 길이가 같아야 한다. 다만 길이가 1인 경우 각각의 원소에 해당 연산을 한다. 즉, x + 3은 x[i]+3을 모든 i에 대해서 계산한 결과를 반환한다.

```
# numpytest1.py
import numpy as np
x = np.array([1,2,3,4])
```

numpy 패키지

```
y= np.array([5,6,7,8])
z= np.array([10,20])
print(x)      ⇨ [1 2 3 4]
print(x*y)     ⇨ [ 5 12 21 32]
print(x + 4)    ⇨  [5 6 7 8]
# print(x+z)  # 에러 길이가 다름
```

np.arange: range 함수와 유사한 np의 메소드로 다음과 같이 사용한다. [start, stop) 사이에 step 만큼의 증분을 가지도록 등차수열이 만들어진다.

```
np.arange([start, ]stop, [step, ... ])
```

start와 step은 생략할 수 있으며 기본값은 각각 0과 1이다. stop은 마지막 수이다. start, stop, step에 float를 사용할 수 있다.

```
print(np.arange(3))    ⇨ [0 1 2]
```

numpy 패키지

print(np.arange(3.0, 7.0)) ⇨ [3. 4. 5. 6.]

print(np.arange(3.1, 4.5, 0.5)) ⇨ [3.1 3.6 4.1]

numpy.linspace

는 np.arange와 유사하나 구간의 간격 step 대신 생성할 배열의 원소의 개수를 설정한다. 사용법은 다음과 같으며

```
numpy.linspace(start=0, stop, num = 50, endpoint = True, retstep=False,...
```

start와 stop은 시작점과 끝나는 점이며 num은 등간격으로 만들 숫자의 개수이다. endpoint는 stop을 포함할지를 설정한다. retstep에는 linspace로 만들어진 간격(step)을 리턴할지 설정한다.

print(np.linspace(2.0, 3.0, num=5)) ⇨ [2. 2.25 2.5 2.75 3.]

print(np.linspace(2.0, 3.0, num=5, retstep=True))

⇨ (array([2. , 2.25, 2.5 , 2.75, 3.]), 0.25)